Intelligenza Artificiale

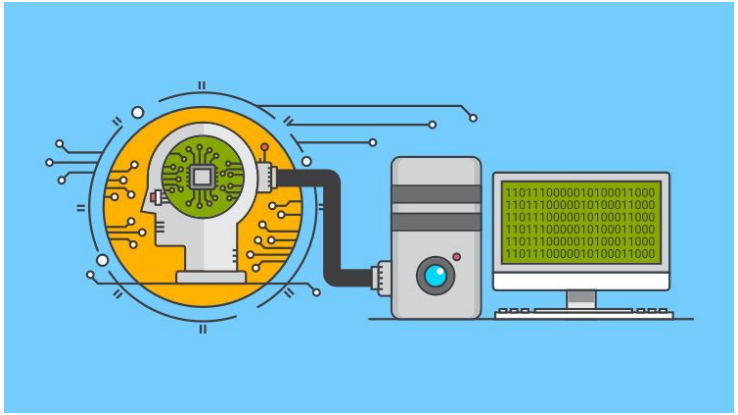
“Machine learning (part 1)”

Nicola Bellotto

A.A. 2025/2026

Lecture outline

- Forms of learning
- Supervised learning
- Model selection and optimization
- Decision trees
- Improvements and extensions



Slides partly based on Russell & Norvig instructors material http://aima.cs.berkeley.edu/instructors.html

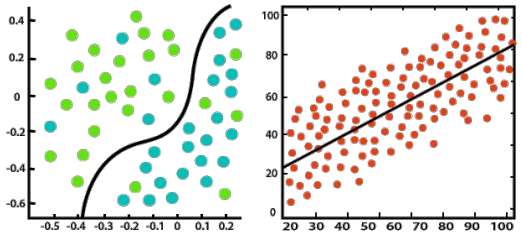
Forms of learning

- Three types of feedback accompanying the inputs → three main types of learning:

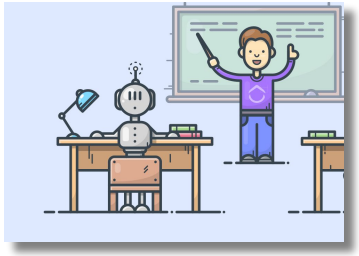
we focus on this

  - **Supervised Learning**
    - agent observes input-output pairs
    - learns map function input → output
  - **Unsupervised Learning**
    - agent learns patterns in the input without any explicit feedback
      - e.g. clustering
  - **Reinforcement Learning**
    - agent learns from series of reinforcements reward/punishment

- Two common learning problems:
  - **Classification**
    - output is one of a finite set of values, e.g. {*cloudy*, *sunny*, *rainy*}
  - **Regression**
    - output is a number, e.g. *pressure*



Supervised learning



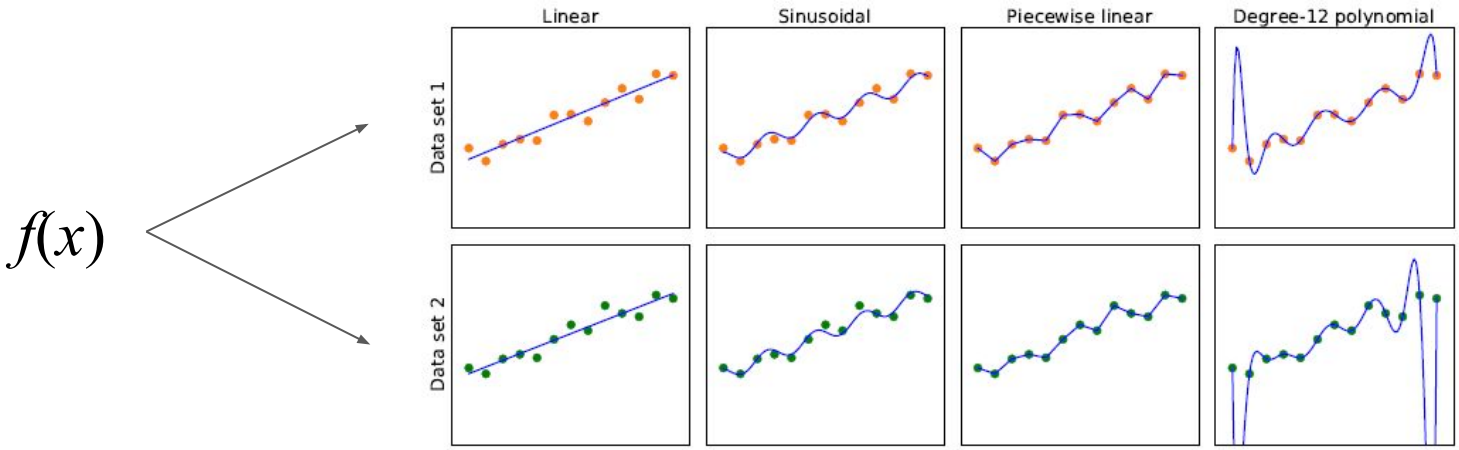
- Supervised algorithms learn from given data of the classification or regression problems

- **Training set** of N examples (*input*, *output*)
  - $(x_1, y_1), (x_2, y_2), \ldots, (x_N, y_N)$

  from some (unknown) process $y = f(x)$
- Output y_i is the so-called **ground truth**

- A function h is a **hypothesis** about the world that approximates the true function f
  - Also called a “model of the data”
  - Generally look for a **best-fit function** so that each $h(x_i)$ is close enough to y_i
- True measure of hypothesis depends on how well it handles inputs not yet seen
  - not with training set, but with a **test set**
- h **generalizes well** if it accurately predicts the outputs of the test set

Supervised learning with different hypotheses

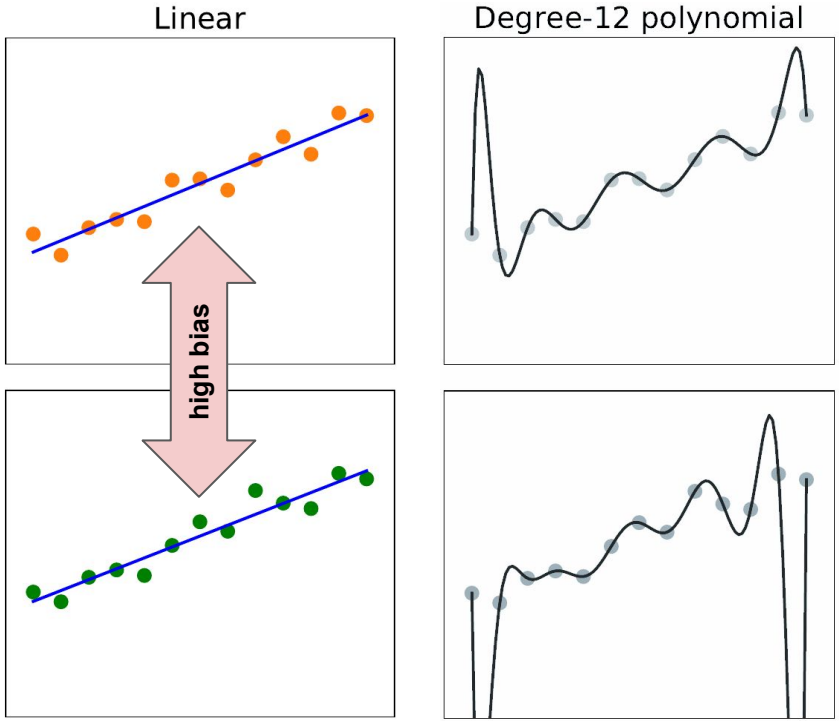


- E.g. finding hypotheses to fit data
  - **Top row**: plots of best-fit functions from four different types of hypothesis trained on a dataset
  - **Bottom row**: same functions but trained on slightly different dataset (sampled from same $f(x)$)

Bias and underfitting

- **Bias**: tendency of a predictive hypothesis to deviate from the expected value when averaged over different training sets
  - E.g. the linear function hypothesis induces a strong bias because is not able to represent non-linear patterns

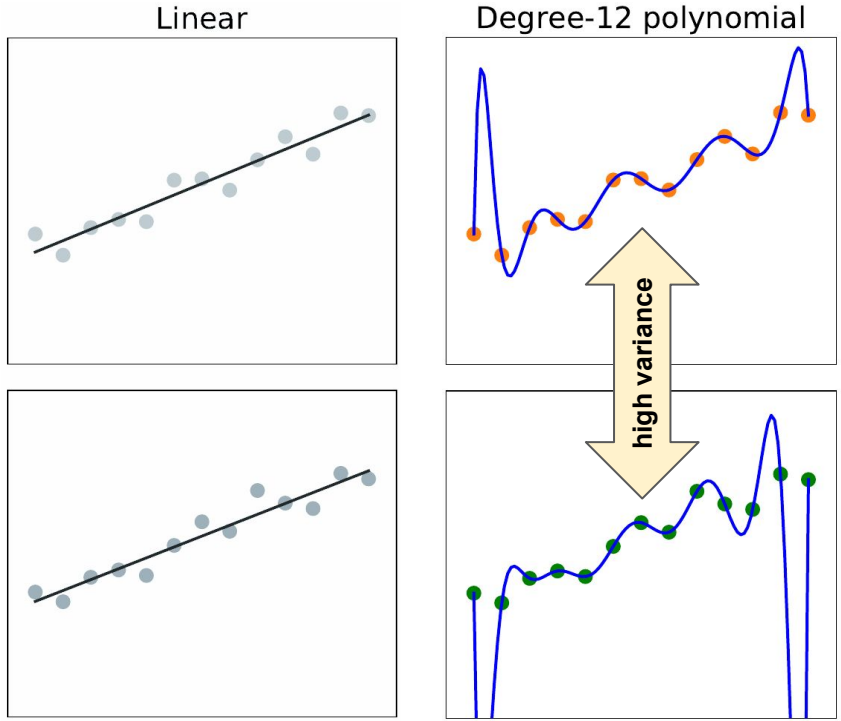
⇒ **Underfitting**: failing to find a pattern in the data



Variance and overfitting

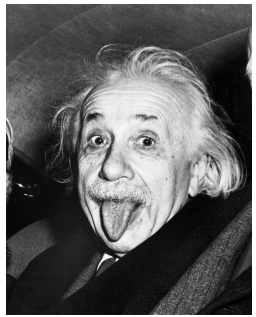
- **Variance**: the amount of change in the hypothesis due to fluctuation in the training data
    - E.g. there is only a small difference in the hypothesis for the linear approximation (low variance), but the two degree-12 polynomials look very different (high variance!) – surely one of the two polynomials must wrong…

⇒ **Overfitting**: paying too much attention to the particular training dataset, causing to perform poorly on unseen data



Bias-variance tradeoff

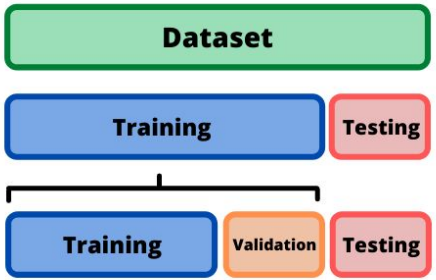
- A choice between more complex, low-bias hypotheses that fit the training data well, and simpler, low-variance hypotheses that may generalize better
  - **Ockham's razor** principle
  - also, "*as simple as possible, but not simpler*" (Albert Einstein)



Model selection and optimization

- Task of finding a good hypothesis has three subtasks:
  - **model selection**: model selection chooses a good hypothesis space (e.g. linear vs polynomial)
  - **optimization** (i.e. **training**) finds the best hypothesis within that space (e.g. parameters of the linear or polynomial approximation)
  - Hypotheses must also be **evaluated** – e.g. check **error rate**: proportion of times that $h(x) \neq y$ for each (x, y)

- Three datasets are needed:
  1. A **training set** to train (create) candidate models
  2. A **validation set** to evaluate and choose best models
  3. A **test set** to do a final unbiased evaluation of the best model

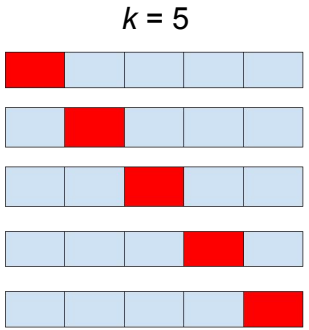


Cross-validation

- If not enough data to create three sets, use ***k*-fold cross-validation**
  - split the data into *k* equal subsets s_i , …, s_k (+ 1 for testing)
  - perform *k* rounds of learning
  - on each round, one subset s_i is held out as a validation set and the remaining examples are used as the training set

- Popular values for *k* are 5 and 10
  - if *k* == size of dataset → called **leave-one-out cross-validation**

- A separate test set is necessary for unbiased assessment

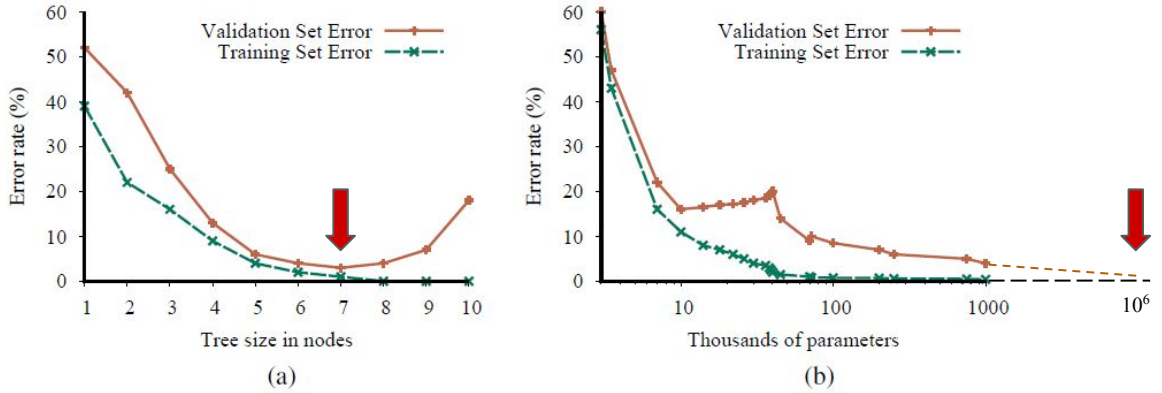


Model evaluation

- Choose the **hyperparameter** value with the lowest **validation-set error**, for example:

  (a) the model class is ***decision trees*** and the hyperparameter is the ***number of nodes*** ⇒ the optimal size is 7

  (b) the model class is ***convolutional neural networks*** and the hyperparameter is the ***number of weights*** in the network ⇒ the optimal number of parameters is 10^6



Example: restaurant problem

- **Problem**: whether to wait or not for a table at the restaurant (classification problem)
- The output y is a Boolean variable *WillWait*
- The input x is a vector of 10 discrete attribute values

1. **Alt**ernative: whether there is a suitable alternative restaurant nearby
2. **Bar**: whether he restaurant has a waiting bar/area
3. **Fri**: true on Fridays and Saturdays
4. **Hun**gry: whether we are hungry right now
5. **Pat**rons: how many customers are in the restaurant – values are {*None*, *Some*, *Full*)}

6. **Price**: restaurant's price range (*$, $$, $$$*)
7. **Rain**: whether it is raining outside
8. **Res**ervation: whether we made a reservation
9. **Type**: the kind of restaurant {*French*, *Italian*, *Thai*, *Burger*}
10. **Est**imated waiting: estimated waiting time: 0-10, 10-30, 30-60, or > 60 minutes

Restaurant training set

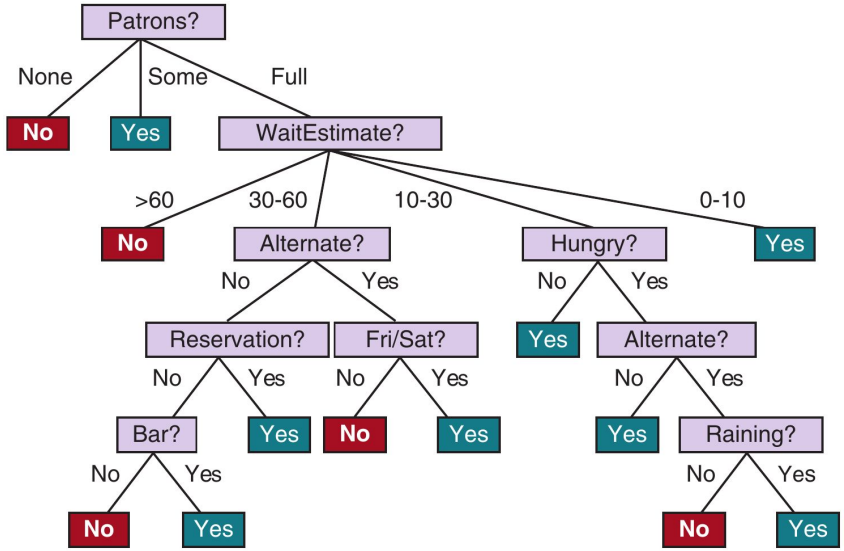
- The dataset is made of 12 previous occasions where we took such a decision

Example	Input Attributes										Output
	Alt	*Bar*	*Fri*	*Hun*	*Pat*	*Price*	*Rain*	*Res*	*Type*	*Est*	*WillWait*
$\mathbf{x}_1$	*Yes*	*No*	*No*	*Yes*	*Some*	*$$$*	*No*	*Yes*	*French*	*0–10*	$y_1 = Yes$
$\mathbf{x}_2$	*Yes*	*No*	*No*	*Yes*	*Full*	*$*	*No*	*No*	*Thai*	*30–60*	$y_2 = No$
$\mathbf{x}_3$	*No*	*Yes*	*No*	*No*	*Some*	*$*	*No*	*No*	*Burger*	*0–10*	$y_3 = Yes$
$\mathbf{x}_4$	*Yes*	*No*	*Yes*	*Yes*	*Full*	*$*	*Yes*	*No*	*Thai*	*10–30*	$y_4 = Yes$
$\mathbf{x}_5$	*Yes*	*No*	*Yes*	*No*	*Full*	*$$$*	*No*	*Yes*	*French*	*>60*	$y_5 = No$
$\mathbf{x}_6$	*No*	*Yes*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Italian*	*0–10*	$y_6 = Yes$
$\mathbf{x}_7$	*No*	*Yes*	*No*	*No*	*None*	*$*	*Yes*	*No*	*Burger*	*0–10*	$y_7 = No$
$\mathbf{x}_8$	*No*	*No*	*No*	*Yes*	*Some*	*$$*	*Yes*	*Yes*	*Thai*	*0–10*	$y_8 = Yes$
$\mathbf{x}_9$	*No*	*Yes*	*Yes*	*No*	*Full*	*$*	*Yes*	*No*	*Burger*	*>60*	$y_9 = No$
$\mathbf{x}_{10}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$$$*	*No*	*Yes*	*Italian*	*10–30*	$y_{10} = No$
$\mathbf{x}_{11}$	*No*	*No*	*No*	*No*	*None*	*$*	*No*	*No*	*Thai*	*0–10*	$y_{11} = No$
$\mathbf{x}_{12}$	*Yes*	*Yes*	*Yes*	*Yes*	*Full*	*$*	*No*	*No*	*Burger*	*30–60*	$y_{12} = Yes$

ground truth

Decision trees

- A **decision tree** is a representation of a function that maps a vector of attribute values to a single output value
  - reaches its decision by performing a sequence of tests, starting at the **root** and following the appropriate branch until a leaf is reached
  - each **internal node** in the tree corresponds to a test of the value of one of the input attributes
  - the **branches** from the node are labeled with the possible values of the attribute
  - **leaf** nodes specify value to be returned by the function (i.e. the **decision**)



Quality of attributes

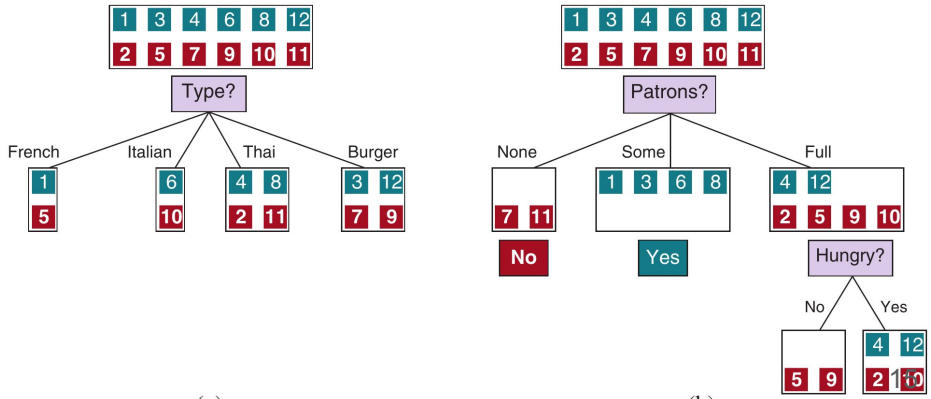
- We could split the dataset examples by testing on attributes:
  - positive (**green boxes**) for which the decision was $WillWait = Yes$
  - negative (**red boxes**) for which the decision was $WillWait = No$

(a) Splitting on *Type* does **not** help us distinguish positive or negative examples

- Not very useful, at least at the beginning…

(b) Splitting on *Patrons* does a better job

- After that, *Hungry* is fairly good…



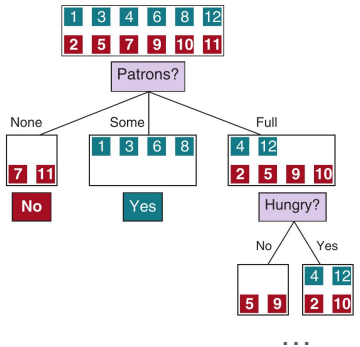
(a) (b)

Learning decision trees

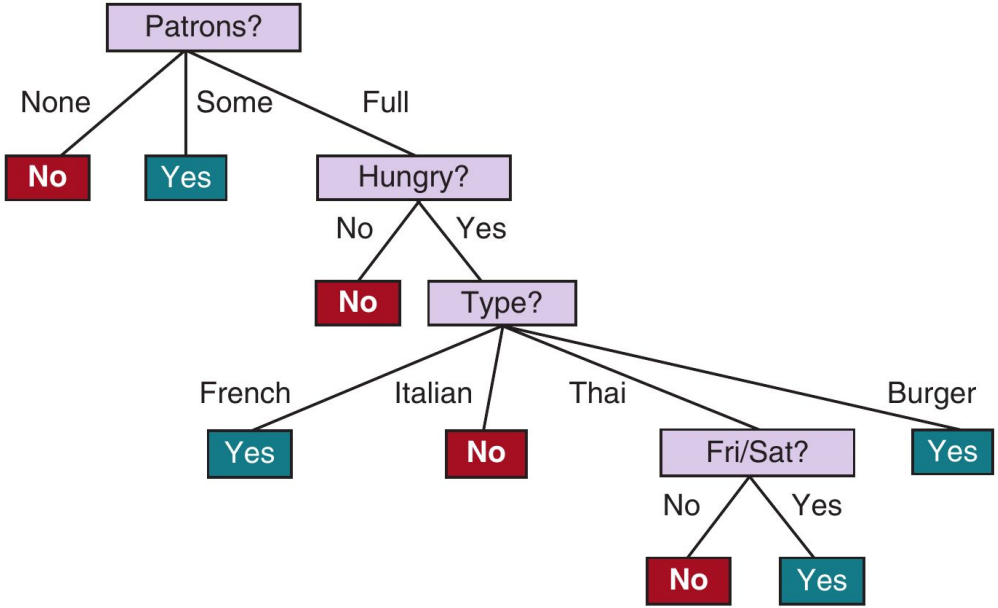
- The decision tree learning algorithm
  - **Aim**: find a small tree consistent with the training examples
  - **Idea**: choose (recursively) most significant attribute as root of (sub)tree
    - choice based on IMPORTANCE of attribute
  - If the resulting examples in leaf have all the same value ⇒ that's the answer! **STOP**
  - Otherwise, PLURALITY-VALUE
    - ➢ if there are no more attributes, select the most common output value among current examples
    - ➢ if there are no more examples, select the most common output value among parents' examples

function LEARN-DECISION-TREE($examples$, $attributes$, $parent\_examples$) **returns** a tree

if $examples$ is empty **then return** PLURALITY-VALUE($parent\_examples$)
else if all $examples$ have the same classification **then return** the classification
else if $attributes$ is empty **then return** PLURALITY-VALUE($examples$)
else
  $A \leftarrow \text{argmax}_{a \in attributes}$ IMPORTANCE(a, $examples$)
  $tree \leftarrow$ a new decision tree with root test A
  for each value v of A **do**
    $exs \leftarrow \{e : e \in examples \text{ and } e.A = v\}$
    $subtree \leftarrow$ LEARN-DECISION-TREE(exs, $attributes - A$, $examples$)
    add a branch to $tree$ with label $(A = v)$ and subtree $subtree$
  return $tree$

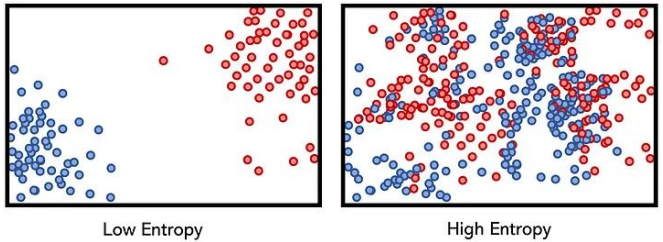


E.g. decision tree after training on restaurant set



Choosing attributes – entropy

- How to evaluate the IMPORTANCE of an attribute?



- **Entropy**: measure of the uncertainty of a random variable
  - fundamental quantity in information theory, measured in ***bits***

- In general, the entropy H of a random variable V with values v_k

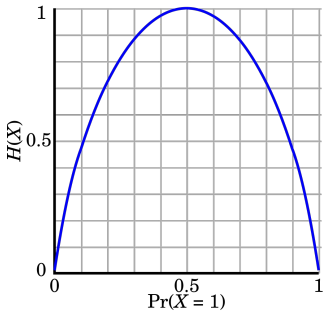
$$H(V) = \sum_k P(v_k) \log_2 \frac{1}{P(v_k)} = -\sum_k P(v_k) \log_2 P(v_k)$$

Choosing attributes – entropy

- E.g. the entropy of a fair coin-toss (50% probability head, 50% tail) is 1 bit

$(P(head)\log_2 1/P(head) + P(tail)\log_2 1/P(tail))$

$= -(0.5\log_2 0.5 + 0.5\log_2 0.5) = \mathbf{1\ bit}$



- Let's call B the **entropy of a Boolean random variable** that is *true* with probability q

$$B(q) = -(q\log_2 q + (1-q)\log_2(1-q))$$

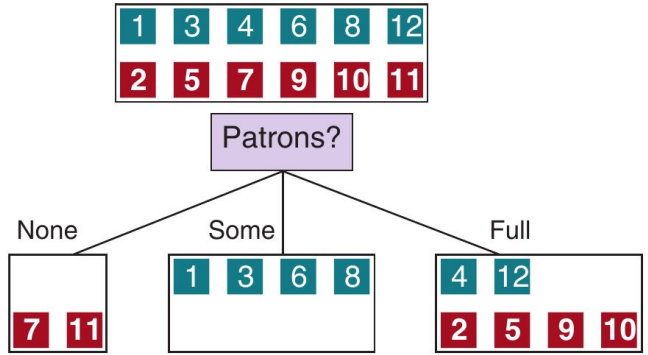
- The probability q can be estimated from the percentage of **positive** examples p across all the examples $N = p + n$ (**positives + negatives**)

$$q = p / N = p / (p + n)$$

$$\Rightarrow \quad B(q) = B(\,p / (p + n)\,)$$

Choosing attributes – expected entropy

- If an attribute A has d possible values, then a test on A will generate d subsets, each one with $N_k = p_k + n_k$ examples
  - E.g. a test on **Patrons** generates three subsets with $N_1 = 0 + 2$, $N_2 = 4 + 0$, and $N_3 = 2 + 4$



- The **expected entropy** after testing on A

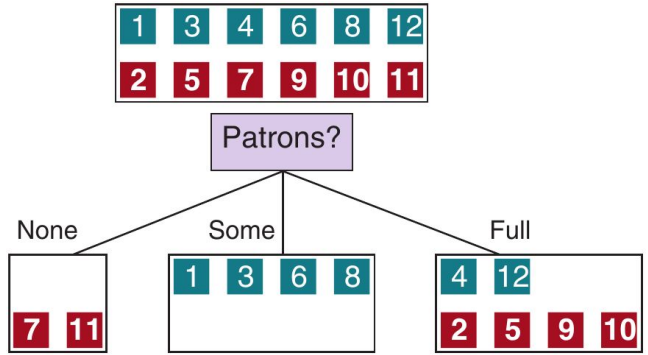
$$Remainder(A) = \sum_{k=1}^{d} \frac{p_k+n_k}{p+n} B\left(\frac{p_k}{p_k+n_k}\right)$$

Choosing attributes – information gain

- **Information gain**: expected reduction in entropy from testing on attribute A

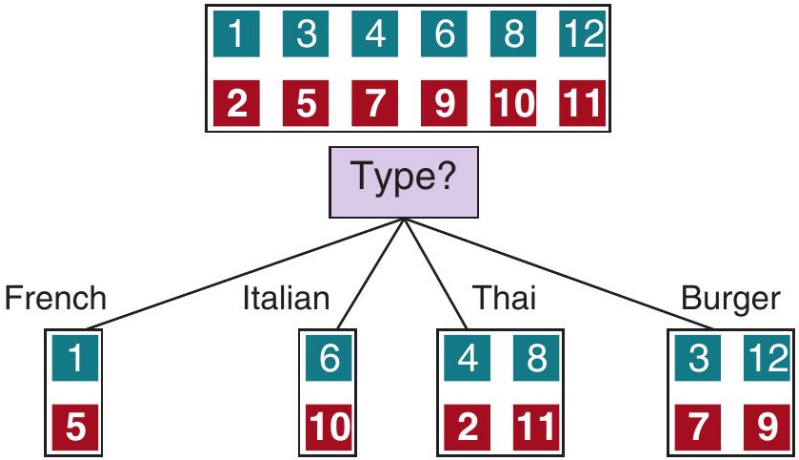
$$Gain(A) = B(\frac{p}{p+n}) - Remainder(A)$$

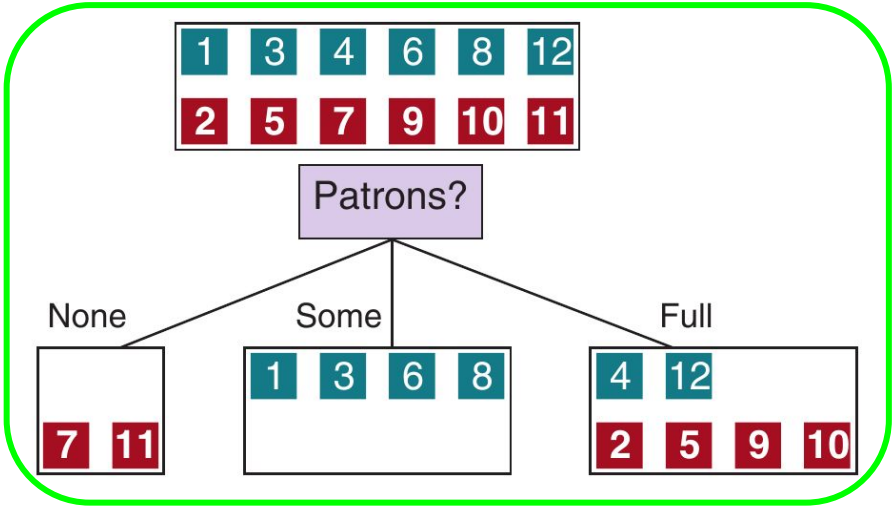
- We want to move from high entropy (sub)sets of training examples to low entropy ones



- Therefore, with IMPORTANCE we choose the attribute that maximises the information gain (i.e. reduce the expected entropy)

Decision trees: choosing attributes





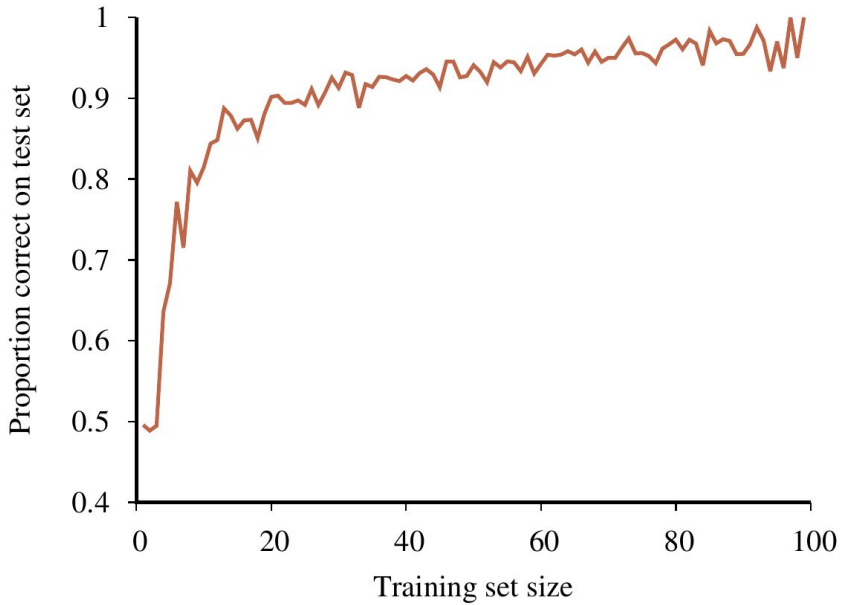
$$Gain(Patrons) = 1 - \left[\tfrac{2}{12}B(\tfrac{0}{2}) + \tfrac{4}{12}B(\tfrac{4}{4}) + \tfrac{6}{12}B(\tfrac{2}{6})\right] \approx 0.541 \text{ bits}$$

$$Gain(Type) = 1 - \left[\tfrac{2}{12}B(\tfrac{1}{2}) + \tfrac{2}{12}B(\tfrac{1}{2}) + \tfrac{4}{12}B(\tfrac{2}{4}) + \tfrac{4}{12}B(\tfrac{2}{4})\right] = 0 \text{ bits}$$

Decision tree performance

- In general the performance increases with the increase of the training set, until a plateau is reached

- **Learning curve** for a decision tree
  - correct answers on the **test set** with varying size of the **training set**

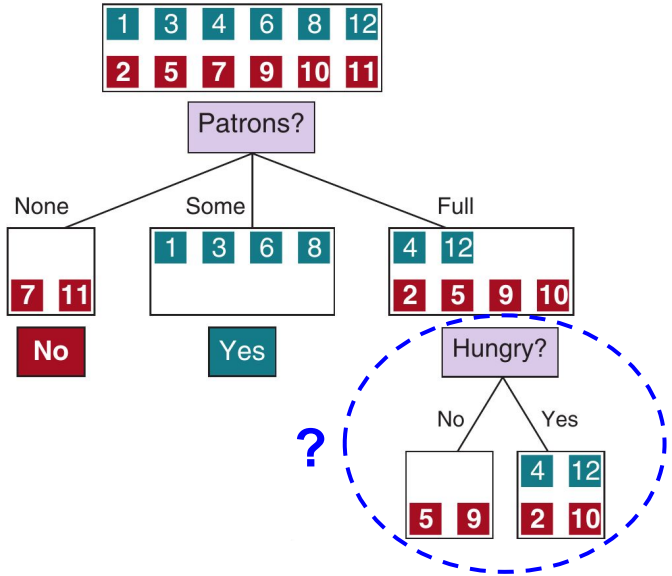


Generalization and overfitting

- More important than fitting the data is the capability of the decision tree, and in general any ML algorithm, to **generalize well** ⇒ perform well on a **test dataset** with examples different from the training set
- As the number of attribute tests grow in a decision tree, so the risk that it will overfit increases (like the degree-12 polynomial in a previous example)
- Several methods to reduce the overfitting problem
  - **Pruning**
  - **Bagging**
  - **Random forest**

Pruning

- **Decision tree pruning** helps combat overfitting
  - Eliminating nodes that are not very relevant
    - Which node? Those with **little** information gain
    - But how little? Depends on a significance test…



- **Significance test**
  - Consider ***null hypothesis***, i.e. attribute is **irrelevant** (information gain →0 for infinite examples)
  - How likely that the null hypothesis is *true*?
    - ➢ if statistically unlikely (typically ≤ 5% probability) ⇒ null hypothesis ***rejected*** ⇒ attribute is useful, keep it
    - ➢ otherwise the null hypothesis is ***confirmed*** and the attributed can be discarded

Pruning

- A common way to test this is by measuring the **total deviation** of the data sample distributions (for the attribute) from those expected under null hypothesis

$$\Delta = \sum_{k=1}^{d} \left[\frac{(p_k - \hat{p}_k)^2}{\hat{p}_k} + \frac{(n_k - \hat{n}_k)^2}{\hat{n}_k} \right]$$

where $\hat{p}_k$ and $\hat{n}_k$ are the expected numbers of positive and negative examples, respectively, for each k value of the attribute, computed as

$$\hat{p}_k = p \times \frac{p_k + n_k}{p + n} \qquad \hat{n}_k = n \times \frac{p_k + n_k}{p + n}$$

or also $\hat{p}_k = p \times N_k / N \qquad \hat{n}_k = n \times N_k / N$

i.e. the expected # of positive or negative samples is proportional to the size of subset k

Pruning

- If $\Delta > \Delta^*$, where Δ^* can be retrieved from a χ^2 (chi-square) distribution table with d–1 **degrees of freedom** for a desired null-hypothesis probability, then we can be reassured that the attribute A is indeed useful
- E.g. *Hungry* attribute

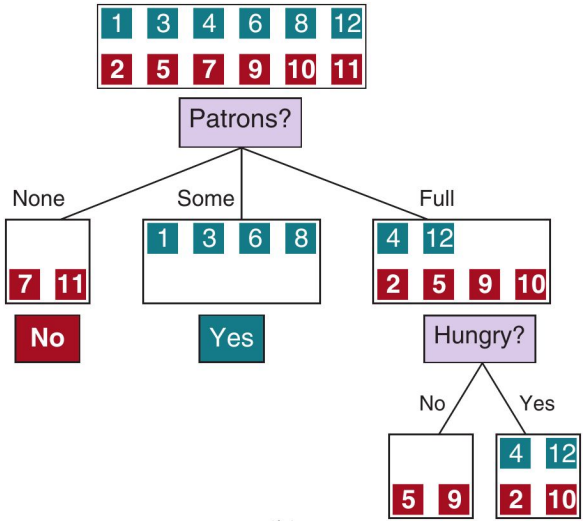
$d = 2 \Rightarrow$ **1** degree of freedom $\Rightarrow \Delta^* = \mathbf{3.841}$ (see next slide)

$\hat{p}_{No} = 2 \times 2 / 6 = \mathbf{0.667}$ $\quad$ $\hat{p}_{Yes} = 2 \times 4 / 6 = \mathbf{1.333}$

$\hat{n}_{No} = 4 \times 2 / 6 = \mathbf{1.333}$ $\quad$ $\hat{n}_{Yes} = 4 \times 4 / 6 = \mathbf{2.667}$

$$\Delta = [(0 - 0.667)^2 / 0.667 + (2 - 1.333)^2 / 1.333] +$$
$$[(2 - 1.333)^2 / 1.333 + (2 - 2.667)^2 / 2.667] = \mathbf{1.5}$$

$\Delta < \Delta^* \Rightarrow$ *Hungry* not very good, we can **prune it**



χ^2 (chi-square) distribution table

l	Probability Content, *p*, between χ^2 and +∞														
	0.995	**0.99**	**0.975**	**0.95**	**0.9**	**0.75**	**0.5**	**0.25**	**0.1**	**0.05**	**0.025**	**0.01**	**0.005**	**0.002**	**0.001**
1	3.927e-5	1.570e-4	9.820e-4	0.00393	0.0157	0.102	0.455	1.323	2.706	3.841	5.024	6.635	7.879	9.550	10.828
2	0.0100	0.0201	0.0506	0.103	0.211	0.575	1.386	2.773	4.605	5.991	7.378	9.210	10.597	12.429	13.816
3	0.0717	0.115	0.216	0.352	0.584	1.213	2.366	4.108	6.251	7.815	9.348	11.345	12.838	14.796	16.266
4	0.207	0.297	0.484	0.711	1.064	1.923	3.357	5.385	7.779	9.488	11.143	13.277	14.860	16.924	18.467
5	0.412	0.554	0.831	1.145	1.610	2.675	4.351	6.626	9.236	11.070	12.833	15.086	16.750	18.907	20.515

https://en.wikibooks.org/wiki/Engineering_Tables/Chi-Squared_Distribution

Bagging

- Sample the training set to generate K distinct training sets, randomly picking N examples (which could be picked more than once)
- Run the algorithm to learn decision trees on the N examples, K times, generating K different decision trees (hypotheses h_i)
- To predict the value of a new input $\mathbf{x}$, we aggregate the predictions from all K hypotheses
  - for **classification**, that means taking the plurality vote (the majority vote for binary classification)
  - for **regression** problems, the final output is the average:

$$h(\mathbf{x}) = \frac{1}{K} \sum_{i=1}^{K} h_i(\mathbf{x})$$

- Bagging reduces variance and is useful to solve problems due to overfitting and/or limited data
- Applicable to any ML model, but mostly used with decision trees

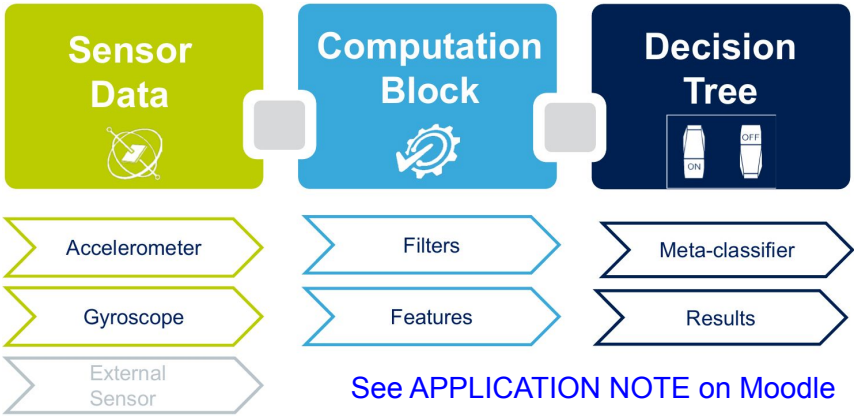
Random forests

- PROBLEM: bagging decision trees can produce K trees that are *highly correlated*
  - If one attribute has very high information gain, likely to be the root of most trees
- **Random forest** (**RF**): a form of decision tree (DT) bagging where the ensemble of K trees is more diverse
- **Key idea**: randomly vary the attribute choices (rather than training examples)
  - At each split point in constructing the tree, randomly select a subset of attributes, and from this choose the one with the highest information gain

- Random forests are efficient to create
  - (a) each split point runs faster because we are considering **fewer attributes**
  - (b) we can **skip the pruning** step because the ensemble as a whole decreases overfitting
  - (c) we could build all the trees in **parallel** on a parallel computer
- While pruning was necessary in DTs to prevent overfitting, RFs are complex, unpruned models resistant to overfitting

Decision tree applications – example

- Integrated into MEMS chips for activity recognition with mobile and wearable devices



Conclusions

- Suggested reading
  - Chapter 19, sec. 19.1-19.4, 19.8 of Russell & Norvig

- Next lecture

  Machine learning (part 2)

- Questions?